DATS301 Final Exam

Hunter Davis

2025-10-22

Dats301 Final Exam

Load The Data (Part 0):

```
data <- read_csv("//DAVISFAMNAS/personal_folder/AMU/DATS301/Rcode/horse_crab.csv")
```

```
## Rows: 173 Columns: 5
## -- Column specification --------------------------------------------------------
## Delimiter: ","
## dbl (5): color, spine, width, satell, weight
##
## i Use `spec()` to retrieve the full column specification for this data.
## i Specify the column types or set `show_col_types = FALSE` to quiet this message.
```

Data Summary (Part 1):

```
colnames(data)
```

```
## [1] "color"  "spine"  "width"  "satell" "weight"
```

```
str(data)
```

```
## spc_tbl_ [173 x 5] (S3: spec_tbl_df/tbl_df/tbl/data.frame)
##  $ color : num [1:173] 4 2 4 4 3 2 4 3 4 4 ...
##  $ spine : num [1:173] 3 1 3 3 3 1 2 1 3 3 ...
##  $ width : num [1:173] 22.5 26 24.8 26 23.8 26.5 24.7 23.7 25.6 24.3 ...
##  $ satell: num [1:173] 0 9 0 4 0 0 0 0 0 0 ...
##  $ weight: num [1:173] 1550 2300 2100 2600 2100 2350 1900 1950 2150 2150 ...
##  - attr(*, "spec")=
##   .. cols(
##   ..   color = col_double(),
##   ..   spine = col_double(),
##   ..   width = col_double(),
##   ..   satell = col_double(),
##   ..   weight = col_double()
##   .. )
##  - attr(*, "problems")=<externalptr>
```

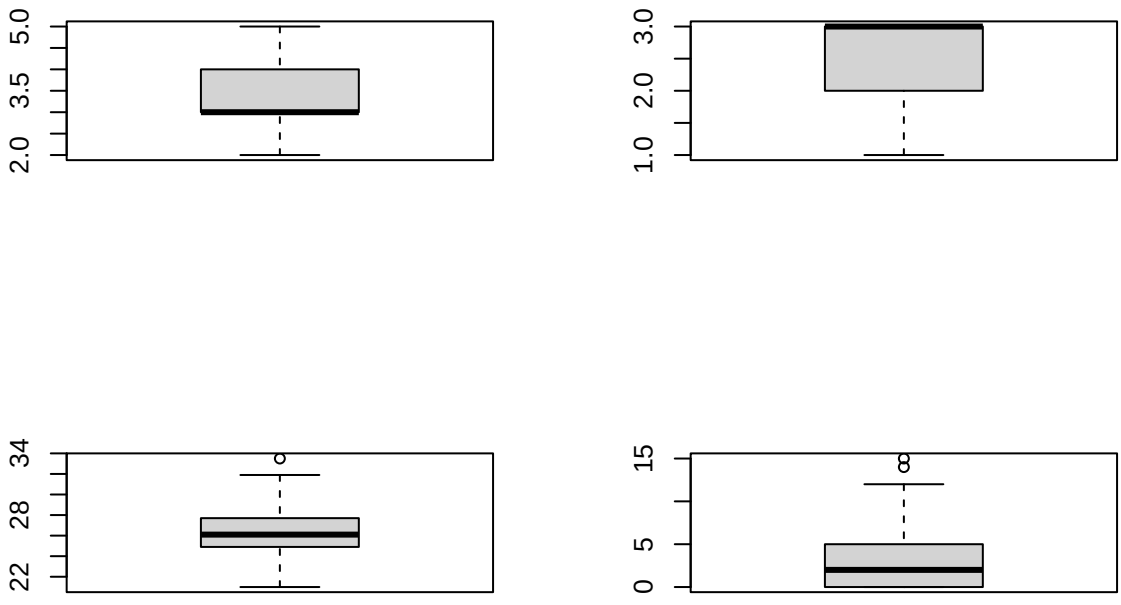
```r
colSums(is.na(data))
```

```
##  color  spine  width satell weight
##      0      0      0      0      0
```

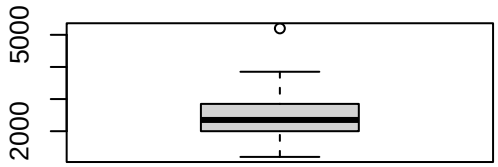
```r
summary(data)
```

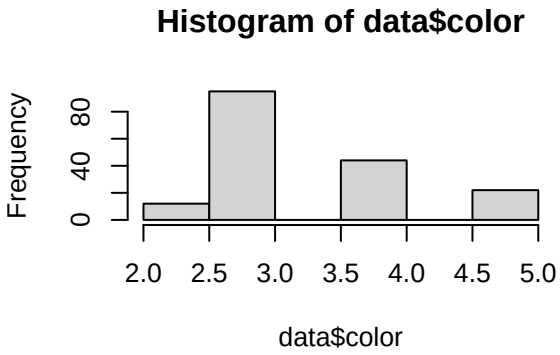
```
##      color           spine           width          satell          weight
##  Min.   :2.000   Min.   :1.000   Min.   :21.0   Min.   : 0.000   Min.   :1200
##  1st Qu.:3.000   1st Qu.:2.000   1st Qu.:24.9   1st Qu.: 0.000   1st Qu.:2000
##  Median :3.000   Median :3.000   Median :26.1   Median : 2.000   Median :2350
##  Mean   :3.439   Mean   :2.486   Mean   :26.3   Mean   : 2.919   Mean   :2437
##  3rd Qu.:4.000   3rd Qu.:3.000   3rd Qu.:27.7   3rd Qu.: 5.000   3rd Qu.:2850
##  Max.   :5.000   Max.   :3.000   Max.   :33.5   Max.   :15.000   Max.   :5200
```

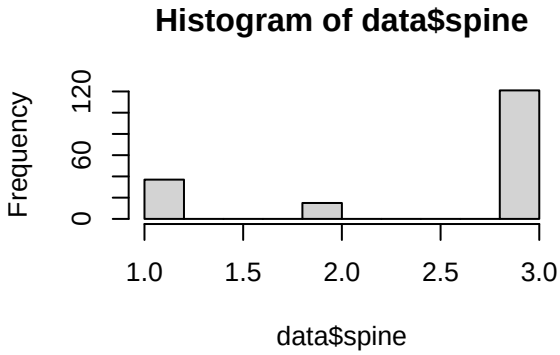
```r
par(mfrow = c(2, 2))

boxplot(data$color)
boxplot(data$spine)
boxplot(data$width)
boxplot(data$satell)
```

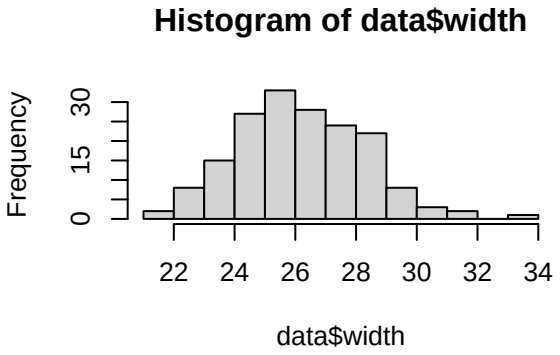


```r
boxplot(data$weight)

hist(data$color)
hist(data$spine)
hist(data$width)
```










```r
hist(data$satell)
hist(data$weight)

data |>
identify_outliers(width)
```

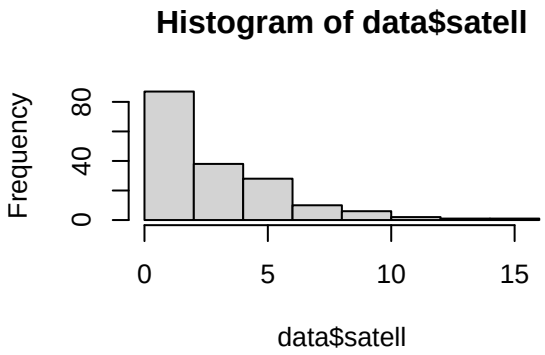
```
## # A tibble: 1 x 7
##   color spine width satell weight is.outlier is.extreme
##   <dbl> <dbl> <dbl>  <dbl>  <dbl> <lgl>      <lgl>
## 1     3     1  33.5      7   5200 TRUE       FALSE
```

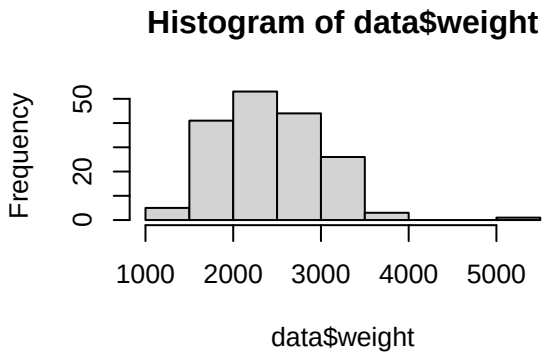
```r
data |>
identify_outliers(satell)
```

```
## # A tibble: 2 x 7
##   color spine width satell weight is.outlier is.extreme
##   <dbl> <dbl> <dbl>  <dbl>  <dbl> <lgl>      <lgl>
## 1     3     1  26       14   2300 TRUE       FALSE
## 2     3     3  28.3     15   3000 TRUE       FALSE
```

```
data |>
identify_outliers(weight)
```

```
## # A tibble: 1 x 7
##   color spine width satell weight is.outlier is.extreme
##   <dbl> <dbl> <dbl>  <dbl>  <dbl> <lgl>      <lgl>
## 1     3     1  33.5      7   5200 TRUE       FALSE
```





From the summary of our data on Horseshoe crabs we can see that we have 173 observation with five different features. All features are of the numeric data type with no missing data values in any of the cells. There is a total of three outliers across the features width, satellite, and weight with one observation appearing in both the width and weight features. The non-categorical features (width, satellite, weight) all have varying degrees of right leaning skewness. Due to the presence of outliers the non-categorical features median values are below the mean value. The width and weight features follow roughly a normal distribution while the satellite distribution is heavily weighted towards the lower counts.

Feature Standardization (Part 2a):

```
data_stand <- data |>
  mutate(width_center=scale(width), weight_center=scale(weight))

head(data_stand)
```

```
## # A tibble: 6 x 7
##   color spine width satell weight width_center[,1] weight_center[,1]
##   <dbl> <dbl> <dbl>  <dbl>  <dbl>            <dbl>             <dbl>
## 1     4     3  22.5      0   1550          -1.80              -1.54
## 2     2     1  26        9   2300          -0.142             -0.238
## 3     4     3  24.8      0   2100          -0.711             -0.584
## 4     4     3  26        4   2600          -0.142              0.282
## 5     3     3  23.8      0   2100          -1.18              -0.584
## 6     2     1  26.5      0   2350           0.0954            -0.151
```

```
summary(data_stand[6:7])
```

```
##    width_center.V1    weight_center.V1
##  Min.   :-2.512419   Min.   :-2.144084
##  1st Qu.:-0.663254   1st Qu.:-0.757663
##  Median :-0.094281   Median :-0.151104
##  Mean   : 0.000000   Mean   : 0.000000
##  3rd Qu.: 0.664351   3rd Qu.: 0.715409
##  Max.   : 3.414390   Max.   : 4.788022
```

Feature standardization is important in data science because it allows us to conduct analysis on data of different scales by adjusting them to have a similar scale for better comparison. Standardization additionally deals with outliers and making them less impact on model performance without the need to eliminate those observations which could lead to lost insights during analysis.

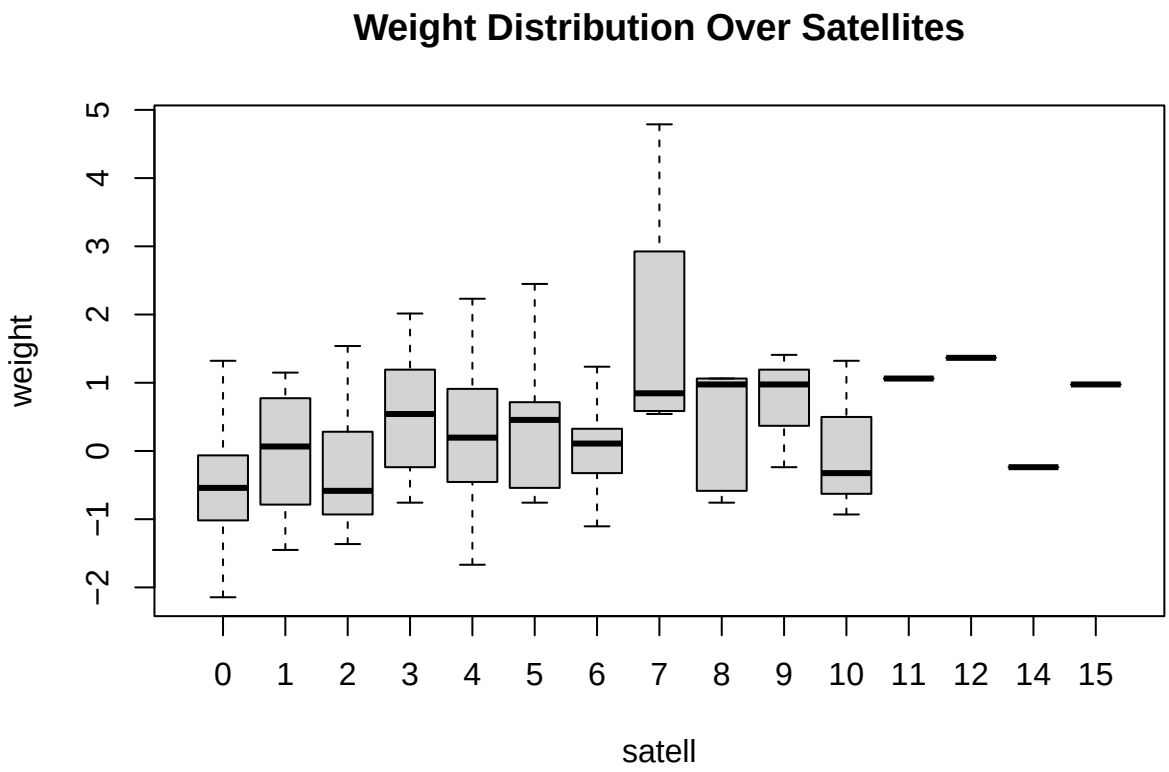
Removing and Renaming features (Part 2b):

```
data_rename <- data_stand |>
  dplyr::select(color, spine, width_center, satell, weight_center) |>
  dplyr::rename(width = width_center, weight = weight_center)

head(data_rename)
```

```
## # A tibble: 6 x 5
##   color spine width[,1] satell weight[,1]
##   <dbl> <dbl>     <dbl>  <dbl>      <dbl>
## 1     4     3   -1.80        0     -1.54
## 2     2     1   -0.142       9     -0.238
## 3     4     3   -0.711       0     -0.584
## 4     4     3   -0.142       4      0.282
## 5     3     3   -1.18        0     -0.584
## 6     2     1    0.0954      0     -0.151
```

Satellite Counts (Part 2c):

```
boxplot(
  weight~satell,
  data = data_rename,
  main='Weight Distribution Over Satellites'
  )
```



From the boxplot we can see that for horseshoe crabs that are lighter in weight tend to have fewer satellites then crabs of heavier weights. This does not mean that the heaviest crabs have the most satellites because we can see that those crabs with the highest satellite counts fall within a narrow range (roughly -0.25 to 1.2 on scaled weight). Given that there is no specific pattern of increasing or decreasing average weight to number of satellites it can be safe to say that crabs falling within a specific weight range have an increased likelihood of having more satellites but are not guaranteed such an outcome. The random spread of the number of satellites to crabs weight leads me to believe that weight has some effect up to a certain point and once a crab passes the upper limit there is less of a chance of increasing the number of satellites. The wide spreads of crabs weights in the lower number of satellites compared to the higher number of satellites indicates a larger number of crabs have fewer satellites while a small part of the population maintains a large number of satellites. If the number of satellites indicates the success of a crab then we could say from this plot that crabs that are too light or too heavy are less successful then those that fall within a specific weight class.

Full Model (Part 3a):

```
set.seed(42)
model_full <- glm(satell~weight*width+color*spine+weight*color+weight*spine+width*color+width*spine,
                  family = 'poisson',
                  data = data_rename
                  )
summary(model_full)
```

```
##
```

```
## Call:
## glm(formula = satell ~ weight * width + color * spine + weight *
##     color + weight * spine + width * color + width * spine, family = "poisson",
##     data = data_rename)
##
## Coefficients:
##              Estimate Std. Error z value Pr(>|z|)
## (Intercept)   2.43805    0.64222   3.796 0.000147 ***
## weight        0.53223    0.49682   1.071 0.284044
## width        -0.68475    0.47103  -1.454 0.146019
## color        -0.41887    0.22183  -1.888 0.058990 .
## spine        -0.34278    0.25254  -1.357 0.174681
## weight:width -0.07270    0.03772  -1.927 0.053937 .
## color:spine   0.10347    0.08270   1.251 0.210870
## weight:color  0.00256    0.16211   0.016 0.987398
## weight:spine -0.04187    0.13982  -0.299 0.764610
## width:color   0.13672    0.14975   0.913 0.361250
## width:spine   0.10207    0.13369   0.763 0.445172
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## (Dispersion parameter for poisson family taken to be 1)
##
##     Null deviance: 632.79  on 172  degrees of freedom
## Residual deviance: 532.19  on 162  degrees of freedom
## AIC: 909.48
##
## Number of Fisher Scoring iterations: 6
```

The full model here shows what a "perfect" model would look like. This captures all the data points observed within the dataset as well as the noise present. This would be a classic overfitting problem where the model performs amazingly on the training data but poorly on test data. This also serves as a benchmark model to compare with simpler models later to determine which model performs best. Additionally we can use the full model dispersion parameter to check for overdispersion.

We can see that all the features of the full model (excluding the intercept) have no statistical significance in predicting the number of satellites. Calculating the deviance dispersion (residual deviance/residual degrees of freedom) we get 3.29 which is greater than 1. This indicates that there is overdispersion, roughly 3.3 times greater then assumed.

Stepwise Selection (Part 3b):

```
stepAIC(model_full,
        direction = "backward",
        trace = TRUE)
```

```
## Start:  AIC=909.48
## satell ~ weight * width + color * spine + weight * color + weight *
##     spine + width * color + width * spine
##
##                Df Deviance    AIC
## - weight:color  1   532.19 907.48
```

```
## - weight:spine  1   532.28 907.57
## - width:spine   1   532.77 908.07
## - width:color   1   533.02 908.32
## - color:spine   1   533.75 909.05
## <none>              532.19 909.48
## - weight:width  1   536.10 911.40
##
## Step:  AIC=907.48
## satell ~ weight + width + color + spine + weight:width + color:spine +
##     weight:spine + width:color + width:spine
##
##                Df Deviance    AIC
## - weight:spine  1   532.29 905.59
## - width:spine   1   532.86 906.16
## - color:spine   1   533.89 907.18
## <none>              532.19 907.48
## - width:color   1   535.50 908.80
## - weight:width  1   536.13 909.43
##
## Step:  AIC=905.59
## satell ~ weight + width + color + spine + weight:width + color:spine +
##     width:color + width:spine
##
##                Df Deviance    AIC
## - width:spine   1   533.44 904.73
## - color:spine   1   533.94 905.24
## <none>              532.29 905.59
## - width:color   1   535.69 906.99
## - weight:width  1   536.58 907.88
##
## Step:  AIC=904.73
## satell ~ weight + width + color + spine + weight:width + color:spine +
##     width:color
##
##                Df Deviance    AIC
## - color:spine   1   534.78 904.08
## <none>              533.44 904.73
## - width:color   1   538.69 907.99
## - weight:width  1   541.01 910.31
##
## Step:  AIC=904.08
## satell ~ weight + width + color + spine + weight:width + width:color
##
##                Df Deviance    AIC
## - spine         1   534.82 902.11
## <none>              534.78 904.08
## - width:color   1   539.29 906.59
## - weight:width  1   542.79 910.09
##
## Step:  AIC=902.11
## satell ~ weight + width + color + weight:width + width:color
##
##                Df Deviance    AIC
## <none>              534.82 902.11
```

```
## - width:color   1   539.29 904.59
## - weight:width  1   542.91 908.21

##
## Call:  glm(formula = satell ~ weight + width + color + weight:width +
##     width:color, family = "poisson", data = data_rename)
##
## Coefficients:
##  (Intercept)        weight         width         color  weight:width
##      1.65200       0.42239      -0.48271      -0.17100      -0.08224
##  width:color
##      0.15385
##
## Degrees of Freedom: 172 Total (i.e. Null);  167 Residual
## Null Deviance:       632.8
## Residual Deviance: 534.8     AIC: 902.1
```

The backward selection starts with the full model and iterates through subsequent models while removing features that reduce the AIC value the most. The first step away from the full model removes the interaction term between weight and color bringing the AIC from 909.48 to 907.48. With a total of five steps away from the full model the AIC drops from 909.48 to 902.11 removing the following features in order of decreasing AIC value:

- weight:color
- weight:spine
- width:spine
- spine

Additionally we can check the dispersion of the final model with the following code to see if the model has improved its fit of the data.

```
model_final <- glm(formula = satell ~ weight + width + color + weight:width + width:color,
                   family = "poisson",
                   data = data_rename)

dispersion <- deviance(model_final) / df.residual(model_final)
print(dispersion)
```

```
## [1] 3.202497
```

Here we see a slight reduction of the dispersion value from 3.29 to 3.20, which which indicates that there is still overdispersion present in the model.

Full vs. Final Model Deviance (Part 3c):

The full model residual deviance (532.19) and degrees of freedom (162) both had an increase in their values (534.82, 167 respectively) for the final model selected with the stepAIC() function. Although there was a change in residual deviance and residual df the dispersion value remained roughly the same which indicates that the model has actually gotten worse at fitting the data.

Full vs. Final AIC (Part 3d):

The full models AIC (909.48) decreases when run through the stepAIC() function resulting in the final model AIC (902.11) indicating that the model that drops the four features (weight:spine, weight:color, width:spine, and spine) provide the optimal balance between model complexity and fit of the data. This doesn't mean that all features selected have statistically significant p-values, just that the final model has the best balance between accuracy and computational requirements. We can see this better by examining the summary of the final model.

```
summary(model_final)
```

```
##
## Call:
## glm(formula = satell ~ weight + width + color + weight:width +
##     width:color, family = "poisson", data = data_rename)
##
## Coefficients:
##              Estimate Std. Error z value Pr(>|z|)
## (Intercept)   1.65200    0.21322   7.748 9.35e-15 ***
## weight        0.42239    0.11118   3.799 0.000145 ***
## width        -0.48271    0.25997  -1.857 0.063342 .
## color        -0.17100    0.06253  -2.735 0.006242 **
## weight:width -0.08224    0.03083  -2.667 0.007646 **
## width:color   0.15385    0.07349   2.093 0.036323 *
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## (Dispersion parameter for poisson family taken to be 1)
##
##     Null deviance: 632.79  on 172  degrees of freedom
## Residual deviance: 534.82  on 167  degrees of freedom
## AIC: 902.11
##
## Number of Fisher Scoring iterations: 6
```

This shows that the width feature isn't strongly significant to the prediction of number of satellites, but because removal would worsen the AIC this is the optimal model when considering a balanced approach.

Removing Width in Final Model (Part 3e):

First off, removing the width feature from the model would also require the removal of the interaction term between width, color, and weight leaving the model only two features (weight and color).

```
model_drop <- glm(formula = satell ~ weight  + color,
                  family = "poisson",
                  data = data_rename)
summary(model_drop)
```

```
##
## Call:
## glm(formula = satell ~ weight + color, family = "poisson", data = data_rename)
##
```

```
## Coefficients:
##             Estimate Std. Error z value Pr(>|z|)
## (Intercept)  1.59177    0.21020   7.573 3.66e-14 ***
## weight       0.31499    0.03894   8.088 6.05e-16 ***
## color       -0.17282    0.06155  -2.808  0.00499 **
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## (Dispersion parameter for poisson family taken to be 1)
##
##     Null deviance: 632.79  on 172  degrees of freedom
## Residual deviance: 552.79  on 170  degrees of freedom
## AIC: 914.09
##
## Number of Fisher Scoring iterations: 6
```

```
dispersion_drop <- deviance(model_drop) / df.residual(model_drop)
print(dispersion_drop)
```

```
## [1] 3.251728
```

Now it's obvious that removing width from the final model wouldn't improve the models performance because it would increase the AIC value from 902.11 to 914.1 which indicates a worsening balance between fit and complexity. Examining the residual deviance and df shows the the dispersion increases as well, to 3.25, indicating that the model has indeed gotten worse at fitting the data. Ultimately the lack of fit that all our models are showing indicates that another type of model would be a better choice for working with the horseshoe crab dataset as it currently exists.

Binary Conversion (Part 4a):

```
summary(data_rename)
```

```
##      color           spine           width.V1          satell
##  Min.   :2.000   Min.   :1.000   Min.   :-2.512419   Min.   : 0.000
##  1st Qu.:3.000   1st Qu.:2.000   1st Qu.:-0.663254   1st Qu.: 0.000
##  Median :3.000   Median :3.000   Median :-0.094281   Median : 2.000
##  Mean   :3.439   Mean   :2.486   Mean   : 0.000000   Mean   : 2.919
##  3rd Qu.:4.000   3rd Qu.:3.000   3rd Qu.: 0.664351   3rd Qu.: 5.000
##  Max.   :5.000   Max.   :3.000   Max.   : 3.414390   Max.   :15.000
##       weight.V1
##  Min.   :-2.144084
##  1st Qu.:-0.757663
##  Median :-0.151104
##  Mean   : 0.000000
##  3rd Qu.: 0.715409
##  Max.   : 4.788022
```

```
data_fact <- data_rename

# Zero is 2 or less satellites, One indicates 3 or more satellites
```

```
data_fact$satell <- factor(ifelse(data_fact$satell<3, 0, 1))

summary(data_fact)
```

```
##      color           spine            width.V1       satell      weight.V1
##  Min.   :2.000   Min.   :1.000   Min.   :-2.512419   0:87   Min.   :-2.144084
##  1st Qu.:3.000   1st Qu.:2.000   1st Qu.:-0.663254   1:86   1st Qu.:-0.757663
##  Median :3.000   Median :3.000   Median :-0.094281          Median :-0.151104
##  Mean   :3.439   Mean   :2.486   Mean   : 0.000000          Mean   : 0.000000
##  3rd Qu.:4.000   3rd Qu.:3.000   3rd Qu.: 0.664351          3rd Qu.: 0.715409
##  Max.   :5.000   Max.   :3.000   Max.   : 3.414390          Max.   : 4.788022
```

```
head(data_fact)
```

```
## # A tibble: 6 x 5
##   color spine width[,1] satell weight[,1]
##   <dbl> <dbl>     <dbl> <fct>       <dbl>
## 1     4     3   -1.80   0          -1.54
## 2     2     1   -0.142  1          -0.238
## 3     4     3   -0.711  0          -0.584
## 4     4     3   -0.142  1           0.282
## 5     3     3   -1.18   0          -0.584
## 6     2     1    0.0954 0          -0.151
```

Since I've converted over the satellite feature into a binary outcome it would be ideal to use machine learning algorithms that deal with binary data. Logistic regression (logit) model is a great starting point, but deals with linear relationships of the log-odds which the crab dataset might not have. Decision tree or random forest models might be the proper starting point if the data is truly non-linear. Finally, if none of these models produce a model that fits the data well XGBoosting or Support Vector Machines (SVM) Model could be the next step, albeit at a greater computational cost.

Binary Full Model (Part 4b):

```
full_mod <- glm(satell ~ weight*width+color*spine+weight*color+weight*spine+width*color+width*spine,
                data = data_fact,
                family = binomial(link = "logit")
                )

summary(full_mod)
```

```
##
## Call:
## glm(formula = satell ~ weight * width + color * spine + weight *
##     color + weight * spine + width * color + width * spine, family = binomial(link = "logit"),
##     data = data_fact)
##
## Coefficients:
##               Estimate Std. Error z value Pr(>|z|)
## (Intercept)   1.007733   2.729413   0.369   0.7120
```

```
## weight        4.035466   2.081586   1.939   0.0525 .
## width        -3.225612   1.924761  -1.676   0.0938 .
## color        -0.205302   0.906283  -0.227   0.8208
## spine        -0.057496   1.051947  -0.055   0.9564
## weight:width -0.118681   0.204933  -0.579   0.5625
## color:spine  -0.004673   0.337360  -0.014   0.9889
## weight:color -0.835012   0.556940  -1.499   0.1338
## weight:spine  0.098073   0.615985   0.159   0.8735
## width:color   0.954582   0.526191   1.814   0.0697 .
## width:spine  -0.184502   0.569802  -0.324   0.7461
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## (Dispersion parameter for binomial family taken to be 1)
##
##     Null deviance: 239.82  on 172  degrees of freedom
## Residual deviance: 201.92  on 162  degrees of freedom
## AIC: 223.92
##
## Number of Fisher Scoring iterations: 5
```

```
dispersion_bino <- deviance(full_mod) / df.residual(full_mod)
print(dispersion_bino)
```

```
## [1] 1.246399
```

Interestingly we have three features that have a slight statistical significance:

- weight (p = 0.0525)
- width (p = 0.0938)
- width:color (p = 0.0697)

The AIC (223.92) and dispersion (1.25) of the logit model is already an improvement over the poisson regression model used earlier in part 3a - 3e. This indicates that this model fits the data much better and that the balance between fit and model complexity has improved.

Backwards Elimination Logit Model (Part 4c):

```
stepAIC(full_mod,
        direction = "backward",
        trace = TRUE)
```

```
## Start:  AIC=223.92
## satell ~ weight * width + color * spine + weight * color + weight *
##     spine + width * color + width * spine
##
##                Df Deviance    AIC
## - color:spine   1   201.92 221.92
## - weight:spine  1   201.94 221.94
## - width:spine   1   202.02 222.02
```

```
## - weight:width  1   202.24 222.24
## <none>              201.92 223.92
## - weight:color  1   204.08 224.08
## - width:color   1   205.29 225.29
##
## Step:  AIC=221.92
## satell ~ weight + width + color + spine + weight:width + weight:color +
##     weight:spine + width:color + width:spine
##
##                Df Deviance    AIC
## - weight:spine  1   201.94 219.94
## - width:spine   1   202.03 220.03
## - weight:width  1   202.24 220.24
## <none>              201.92 221.92
## - weight:color  1   204.27 222.27
## - width:color   1   205.42 223.42
##
## Step:  AIC=219.94
## satell ~ weight + width + color + spine + weight:width + weight:color +
##     width:color + width:spine
##
##                Df Deviance    AIC
## - width:spine   1   202.12 218.12
## - weight:width  1   202.31 218.31
## <none>              201.94 219.94
## - weight:color  1   204.37 220.37
## - width:color   1   205.57 221.57
##
## Step:  AIC=218.12
## satell ~ weight + width + color + spine + weight:width + weight:color +
##     width:color
##
##                Df Deviance    AIC
## - spine         1   202.25 216.25
## - weight:width  1   202.49 216.49
## <none>              202.12 218.12
## - weight:color  1   204.68 218.68
## - width:color   1   205.60 219.60
##
## Step:  AIC=216.25
## satell ~ weight + width + color + weight:width + weight:color +
##     width:color
##
##                Df Deviance    AIC
## - weight:width  1   202.59 214.59
## <none>              202.25 216.25
## - weight:color  1   204.73 216.73
## - width:color   1   205.64 217.64
##
## Step:  AIC=214.58
## satell ~ weight + width + color + weight:color + width:color
##
##                Df Deviance    AIC
## <none>              202.59 214.59
```

```
## - weight:color  1   205.06 215.06
## - width:color   1   206.41 216.41

##
## Call:  glm(formula = satell ~ weight + width + color + weight:color +
##     width:color, family = binomial(link = "logit"), data = data_fact)
##
## Coefficients:
##  (Intercept)        weight         width         color  weight:color
##       0.8792        4.1808       -3.5855       -0.2458       -0.8099
##  width:color
##       0.9225
##
## Degrees of Freedom: 172 Total (i.e. Null);  167 Residual
## Null Deviance:       239.8
## Residual Deviance: 202.6     AIC: 214.6
```

After six iterations the stepAIC() function has reduced the AIC value from 223.92 to 214.6. This outcome is achieved by removing the following features:

- color:spine
- weight:spine
- width:spine
- spine
- weight:width

This leaves us with a mode that only contain the features that create the optimal balance between fit and model complexity (weight, width, color, width:color, weight:color).

Goodness of fit test (Part 4d):

```r
reduced_model <- glm(satell ~ weight + width + color + weight:color + width:color,
                     family = binomial(link = "logit"),
                     data = data_fact
                     )

# Perform the Likelihood Ratio Test
lrt_result <- anova(reduced_model, full_mod, test = "LRT")

# Print the result
print(lrt_result)
```

```
## Analysis of Deviance Table
##
## Model 1: satell ~ weight + width + color + weight:color + width:color
## Model 2: satell ~ weight * width + color * spine + weight * color + weight *
##     spine + width * color + width * spine
##   Resid. Df Resid. Dev Df Deviance Pr(>Chi)
## 1       167     202.59
## 2       162     201.92  5  0.66813   0.9847
```

After examining the output of the Likelihood ratio test to see which model would be the better choice. The p-value of the full model (0.9847) is much larger then the critical value (a = 0.05) which indicates that there is not significant evidence that the full model is better then the reduced model. Based on this evidence it would be prudent to use the reduced model moving forward in the analysis.

AIC Comparison (Part 4e):

```
cat("Full Model AIC: \t", full_mod$aic)
```

```
## Full Model AIC:   223.9166
```

```
cat("\nReduced Model AIC: \t", reduced_model$aic)
```

```
##
## Reduced Model AIC:    214.5847
```

It's clear that based on AIC values between the two models that the reduced model being 9 units smaller has the more ideal performance.

Selected Model Comparison 3b and 4b (Part 4f):

```
# Perform the Likelihood Ratio Test
lrt_result2 <- anova(model_final, reduced_model, test = "LRT")

# Print the result
print(lrt_result2)
```

```
## Analysis of Deviance Table
##
## Model 1: satell ~ weight + width + color + weight:width + width:color
## Model 2: satell ~ weight + width + color + weight:color + width:color
##   Resid. Df Resid. Dev Df Deviance Pr(>Chi)
## 1       167     534.82
## 2       167     202.58  0   332.23
```

```
cat("\n3b Model AIC: \t", model_final$aic)
```

```
##
## 3b Model AIC:     902.1147
```

```
cat("\n4b Model AIC: \t", reduced_model$aic)
```

```
##
## 4b Model AIC:     214.5847
```

Yet again when comparing the two final models that are selected using the stepAIC() function for the different models in part 3b and 4b shows that the 4b model would be preferable compared to the 3b model if we solely base our decision on the AIC value. Interestingly enough both models have the same number of features (5) with a slight variation in the interaction variables (3b: weight x width, and 4b: weight x color). The LRT result really shouldn't be used here to compare but it helps visualize what the models features are in a condense format.

Table Questions (Part 5):

```
kable(table(data_rename$color, data_rename$spine))
```

	1	2	3
2	9	2	1
3	24	8	63
4	3	4	37
5	1	1	20

Question 1:

What is the probability of a having color 3 given that the related spine is 1. We're looking for the probability that the crabs color is "color 3" when they are also "spine 1". P(color = 3| spine = 1).

```
# Total number of crabs with spine = 1.
spine1 <-  9 + 24 + 3 + 1

# Value of crabs with color = 3 and also spine = 1
col3_spine1 <- 24

# P(color = 3| spine = 1)
prob_col3_spine1 <- col3_spine1 / spine1

cat("The probability that a horseshoe crab has the color
    designation 3 given they're spine designation 1 is: ",
    round(prob_col3_spine1, 4)*100,"%")
```

```
## The probability that a horseshoe crab has the color
##     designation 3 given they're spine designation 1 is:  64.86 %
```

Question 2:

What is the probability of having a spine of 2 given that related color is 4. P(spine = 2| color = 4).

```
# Total number of crabs with color = 4.
col4 <-  3 + 4 + 37

# Value of crabs with spine = 2 and also color = 4
spine2_col4 <- 4

# P(color = 3| spine = 1)
prob_spine2_col4 <- spine2_col4 / col4

cat("The probability that a horseshoe crab has the spine
    designation 2 given they're color designation 4 is: ",
    round(prob_spine2_col4, 4)*100,"%")
```

```
## The probability that a horseshoe crab has the spine
##     designation 2 given they're color designation 4 is:  9.09 %
```

Equivalency (Part 6):

Answer on following page.

Show that the two given equations are equivalent.

Given Equations: $log\left(\frac{\pi}{1-\pi}\right) = X\beta,\ \ \pi = \frac{1}{1+e^{-X\beta}}$

Exponentiate both sides: $e^{\left(log\left(\frac{\pi}{1-\pi}\right)\right)} = e^{(X\beta)}$

$$\frac{\pi}{1-\pi} = e^{(X\beta)}$$

Solve for π: $\pi = e^{(X\beta)}(1-\pi)$

$$\pi = \left(e^{(X\beta)} - \pi e^{(X\beta)}\right)$$

$$\pi + \pi e^{(X\beta)} = e^{(X\beta)}$$

$$\pi\left(1 + e^{(X\beta)}\right) = e^{(X\beta)}$$

$$\pi = \frac{e^{(X\beta)}}{(1 + e^{(X\beta)})}$$

$$\pi = \frac{e^{(X\beta)} * e^{(-X\beta)}}{(1 + e^{(X\beta)}) * e^{(-X\beta)}}$$

$$\pi = \frac{1}{e^{-X\beta} + 1} = \frac{1}{1 + e^{-X\beta}}$$

$$log\left(\frac{\pi}{1-\pi}\right) = X\beta \Leftrightarrow \pi = \frac{1}{1 + e^{-X\beta}}$$

Figure 1: Equivalent Check